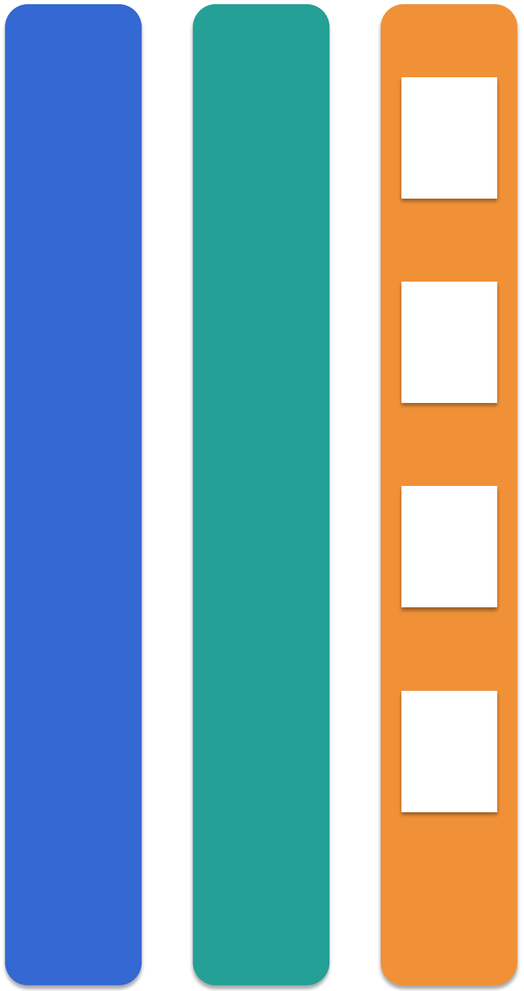
RV College of Engineering®

UNIT-II

STACKS

BY
Dr. Praveena T
Associate Professor
R V College of Engineering
Bengaluru.



Go, change the world®

Learning Objectives

By the end of the unit, students should be able to…

- ✔ Explain the LIFO principle and Stack ADT.
- ✔ Perform PUSH, POP and PEEK operations correctly.
- ✔ Implement stacks using arrays and linked lists.
- ✔ Analyze overflow, underflow and operation complexity.
- ✔ Apply stacks to expression conversion and evaluation.
- ✔ Use stacks for parenthesis balancing and understand Tower of Hanoi.

Why Do We Need Stacks?

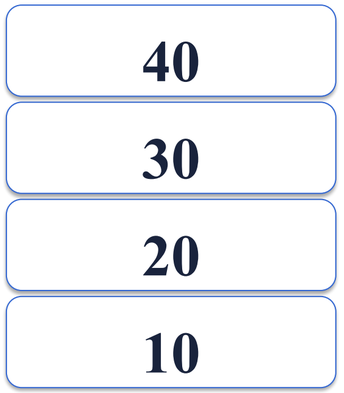
Motivation and real-world relevance

- ✔ Many problems require access to the most recently added item first.
- ✔ Stacks naturally model nested and reversible activities.
- ✔ Examples: undo/redo, browser back button, function calls, recursion, syntax checking.
- ✔ The restriction to one access end makes stack operations simple and efficient.

LIFO Principle

Core discipline of a stack

- ✔ LIFO = Last-In, First-Out.
- ✔ Insertion and deletion occur only at the TOP.
- ✔ The last element pushed is the first element popped.
- ✔ Analogy: a pile of plates — the top plate is removed first.



POP

40 is removed first

What vs How

- ✔ An Abstract Data Type defines the data behaviour and supported operations, not the implementation.
- ✔ Typical operations: PUSH, POP, PEEK/TOP, isEmpty and isFull.
- ✔ The same ADT can be implemented using an array or a linked list.
- ✔ ADT improves modularity because client code depends on operations rather than representation.

Stack Terminology

Essential vocabulary

- ✔ TOP: position/reference to the most recently inserted element.
- ✔ PUSH: insertion at TOP.
- ✔ POP: deletion from TOP.
- ✔ Overflow: insertion attempted when a fixed stack is full.
- ✔ Underflow: deletion/access attempted when the stack is empty.

PUSH Operation

Algorithm

- ✔ Step 1: Check whether the stack is full (array implementation).
- ✔ Step 2: Increment TOP.
- ✔ Step 3: Store the new item at stack[TOP].
- ✔ Time complexity: O(1).

POP Operation

Algorithm

- ✔ Step 1: Check whether the stack is empty.
- ✔ Step 2: Read stack[TOP].
- ✔ Step 3: Decrement TOP.
- ✔ Step 4: Return the removed element.
- ✔ Time complexity: O(1).

PEEK / TOP Operation

Non-destructive access

- ✔ PEEK returns the element currently at TOP without deleting it.
- ✔ Useful when an algorithm needs to inspect the next item to process.
- ✔ On an empty stack, PEEK causes an underflow condition.
- ✔ Time complexity: O(1).

Overflow and Underflow

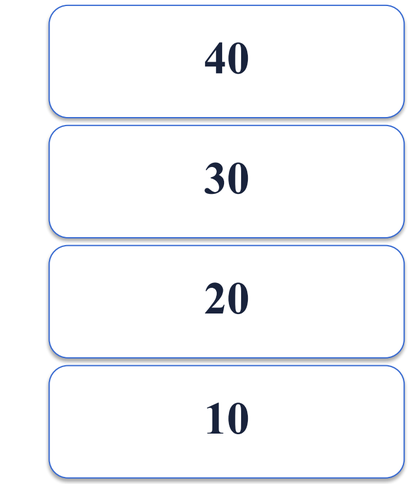
Error conditions

- ✔ Overflow occurs when PUSH is performed on a full fixed-size stack.
- ✔ Underflow occurs when POP or PEEK is performed on an empty stack.
- ✔ Array stack: full when TOP = MAX − 1.
- ✔ Empty stack: TOP = −1 (common convention).

Array Implementation

Static implementation

- ✔ Use an array stack[MAX] and an integer TOP.
- ✔ Initially TOP = −1.
- ✔ Array representation is simple and cache-friendly.
- ✔ Main limitation: maximum capacity is fixed at creation time.



TOP
top = 3

Linked-List Implementation


```
#define MAX 100
int stack[MAX], top = -1;

void push(int x) {
    if (top == MAX-1) printf("Overflow");
    else stack[++top] = x;
}

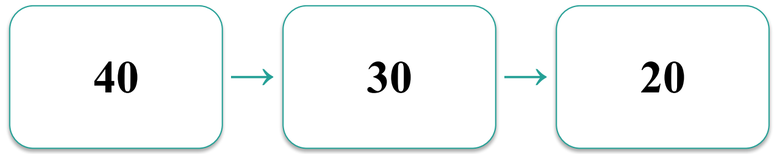
int pop(void) {
    if (top == -1) return -1;
    return stack[top--];
}
```

- ✔ Use a fixed-size array and a TOP variable.
- ✔ PUSH and POP both update TOP.
- ✔ Check boundary conditions before accessing the array.
- ✔ This implementation is suitable when maximum size is known.

Array Stack – C Implementation

Dynamic implementation

- ✔ Each node contains data and a link to the next node.
- ✔ TOP points to the first node.
- ✔ PUSH inserts a node at the beginning.
- ✔ POP removes the first node.
- ✔ Capacity grows dynamically until memory is unavailable.



Array vs Linked List

Comparison

ARRAY

Fixed size
Contiguous memory
Low overhead

LINKED LIST

Dynamic size
Node + pointer
Flexible capacity

BOTH

PUSH/POP O(1)
when TOP is maintained

Applications of Stacks

Where stacks are used

- ✔ Expression conversion and evaluation.
- ✔ Balanced parentheses and syntax checking.
- ✔ Function calls and recursion.
- ✔ Undo/redo mechanisms.
- ✔ Depth-first search and backtracking.
- ✔ Browser navigation and history.

Expression Notations

Arithmetic expressions

- ✔ Infix: A + B — operator between operands.
- ✔ Prefix: + A B — operator before operands.
- ✔ Postfix: A B + — operator after operands.
- ✔ Postfix and prefix avoid the need for parentheses during evaluation.

Operator Precedence & Associativity

Rules needed for conversion

- ✔ Parentheses have highest control over grouping.
- ✔ Exponentiation generally has higher precedence than multiplication/division.
- ✔ Multiplication/division precede addition/subtraction.
- ✔ Associativity determines evaluation order when precedence is equal.

HIGH

( )

MEDIUM

^ then × ÷

LOW

+ −

Infix → Postfix: Basic Algorithm

Stack-based conversion

- ✔ Scan the expression from left to right.
- ✔ Operand → add directly to output.
- ✔ Opening parenthesis → PUSH.
- ✔ Closing parenthesis → POP until opening parenthesis is found.
- ✔ Operator → POP higher/equal-precedence operators as appropriate, then PUSH current operator.
- ✔ After scanning, POP remaining operators.

Worked Conversion

Step-by-step example

- ✔ Expression: A + B * C
- ✔ Read A → output A
- ✔ Read + → PUSH +
- ✔ Read B → output AB
- ✔ Read * → PUSH * because it has higher precedence
- ✔ Read C → output ABC
- ✔ End → POP * then +
- ✔ Postfix: A B C * +

RESULT

A B C * +

WHY?

* has higher precedence than +

Postfix Evaluation

Evaluation algorithm

- ✔ Scan tokens left to right.
- ✔ Operand → PUSH onto stack.
- ✔ Operator → POP right operand, then left operand.
- ✔ Apply the operator and PUSH the result.
- ✔ After the final token, the stack contains the result.

Postfix Evaluation – Trace

Worked example

- ✔ Expression: 2 3 4 * +
- ✔ 2 → [2]
- ✔ 3 → [2, 3]
- ✔ 4 → [2, 3, 4]
- ✔ * → 3×4 = 12 → [2, 12]
- ✔ + → 2+12 = 14 → [14]
- ✔ Result = 14

14

FINAL RESULT

14

Parenthesis Balancing

Balanced-symbol algorithm

- ✔ Push every opening bracket: (, {, [.
- ✔ For a closing bracket, the stack must not be empty.
- ✔ The TOP must be the matching opening bracket.
- ✔ Pop after a successful match.
- ✔ After scanning the whole expression, the stack must be empty.
- ✔ Time complexity: O(n).

Parenthesis Examples

Practice classification

- ✔ Balanced: { [(A + B)] }
- ✔ Balanced: (A + B) * [C − D]
- ✔ Not balanced: { [(A + B]) }
- ✔ Not balanced: (A + B
- ✔ Not balanced: A + B)

Tower of Hanoi

Classic recursive application

- ✔ Move n disks from source peg to destination peg using an auxiliary peg.
- ✔ Move only one disk at a time.
- ✔ Never place a larger disk on a smaller disk.
- ✔ The recursive solution naturally creates activation records on the call stack.
- ✔ Minimum number of moves: $2^n - 1$.

Tower of Hanoi – Algorithm

Recursive solution

```
Hanoi(n, source, auxiliary, destination)

if n == 1:
    move source → destination
else:
    Hanoi(n−1, source, destination, auxiliary)
    move source → destination
    Hanoi(n−1, auxiliary, source, destination)
```

RECURRENCE

$T(n) = 2T(n-1) + 1$

MOVES

$2^n - 1$

Tower of Hanoi – Example

Understanding the recursion

- ✔ For n = 3: minimum moves = $2^3 - 1 = 7$.
- ✔ The recursion breaks the problem into two smaller n−1 problems.
- ✔ Each recursive call adds an activation record to the call stack.
- ✔ This demonstrates how stacks support recursive execution.

n = 3

$2^3 - 1 = 7$ moves

CALL STACK

Each recursive call waits for smaller subproblem to finish.

Complexity Summary

Important exam point

PUSH: O(1)

POP: O(1)

PEEK: O(1)

isEmpty / isFull: O(1)

Infix → Postfix: O(n)

Postfix evaluation: O(n)

Parenthesis balancing: O(n)

Tower of Hanoi: $O(2^n)$ moves

Debugging checklist

- ✔ Forgetting to check overflow before PUSH.
- ✔ Forgetting to check underflow before POP.
- ✔ Popping operands in the wrong order during subtraction/division.
- ✔ Ignoring operator precedence during conversion.
- ✔ Failing to verify that the final parenthesis stack is empty.
- ✔ Confusing the TOP element with the bottom element.

Classroom Practice

Think • Solve • Discuss

1

Convert: A + B * C − D to postfix.

2

Evaluate: 2 5 3 * +.

3

Determine whether {[()]} is balanced.

4

Determine the minimum moves for 4 Hanoi disks.

5

Write PUSH and POP algorithms for an array stack.

Unit Summary

Key takeaways

- ✔ Stack follows LIFO.
- ✔ TOP is the only end used for insertion and deletion.
- ✔ Stacks can be implemented with arrays or linked lists.
- ✔ Stacks are central to expression processing and syntax checking.
- ✔ Recursion relies on the call stack.
- ✔ Mastering stack operations provides a foundation for queues, trees, graphs and algorithms.

THANK YOU